Golang Intern Task Assignment

TaskBridge: Cross-Platform Remote Job Runner

Project Goal

Build a small Go-based system where a server accepts job requests and one or more agents execute those jobs and report results back.

Use of LLM

Usage of LLM to streamline and expedite your application building workflow is encouraged, but deep understanding of why a certain architectural or procedural decision was made during the development is required.

System Components

Component	Responsibility	Example Binary
Server	Accepts jobs, tracks lifecycle, stores agents, assigns jobs, receives results.	taskbridge-server
Agent	Registers with server, sends heartbeat, polls for jobs, executes safe job types, reports result/logs.	taskbridge-agent

Core APIs

- POST /jobs - create a job.
- GET /jobs - list jobs.
- GET /jobs/{jobId} - fetch one job.
- POST /agents/register - register an agent.
- POST /agents/{agentId}/heartbeat - update agent liveness.
- POST /agents/{agentId}/next-job - assign the next compatible pending job.
- POST /jobs/{jobId}/result - submit job result, logs, and error information.
- GET /agents - list active and inactive agents.
- GET /health - basic server health check.

Example Job Request

```
{
  "name": "check-app-health",
  "type": "http_check",
  "payload": {
    "url": "http://localhost:8080/health",
    "expected_status": 200
  },
  "timeout_seconds": 10,
  "max_retries": 2
}
```

Job Lifecycle

- PENDING -> RUNNING -> SUCCESS
- PENDING -> RUNNING -> FAILED
- PENDING -> RUNNING -> RETRYING -> RUNNING -> SUCCESS or FAILED

Each job should store: job_id, name, type, payload, status, assigned_agent_id, created_at, started_at, finished_at, attempt_count, max_retries, logs, error, and result.

Supported Job Types

Job Type	Purpose
http_check	Check whether an HTTP endpoint returns an expected status code.
tcp_check	Check whether a TCP address is reachable.
file_exists	Check whether a file exists at a given path.
checksum	Calculate checksum for a file.
copy_file	Copy a file from one location to another.
write_file	Write controlled content to a file.
wait	Wait for a fixed duration, useful for testing retries and timeouts.
command	Optional stretch goal only; should be disabled or strictly allowlisted by default.

Milestones

1. Basic server: create jobs, list jobs, get job by ID, and expose health endpoint using in-memory storage.
2. Basic agent: register with server, poll for pending jobs, execute http_check jobs, and report results.
3. Job lifecycle and retries: implement PENDING, RUNNING, SUCCESS, FAILED, RETRYING, timeout_seconds, max_retries, and attempt_count.
4. Multiple job types: add safe handlers such as tcp_check, file_exists, checksum, write_file, and wait.
5. Agent heartbeat and status: track last_seen and mark agents online/offline based on heartbeat freshness.
6. Testing and documentation: add unit tests, README, API examples, curl commands, and sample payloads.

Stretch Goals

- SQLite persistence for jobs, agents, attempts, and logs.
- WebSocket mode so the server can push jobs to connected agents.
- Simple dashboard showing jobs, agents, status, failures, and logs.
- Job cancellation endpoint and cooperative cancellation in the agent.
- Capability-based scheduling for assigning jobs only to compatible agents.
- Shared auth token for agent registration and job polling.

Evaluation Criteria

*All have equal weightage

Area	Expectations
Go fundamentals	Clean structs, interfaces, packages, naming, and error handling.
API design	Clear REST endpoints and JSON request/response structures.
Concurrency	Safe in-memory state using mutexes or channels.

Reliability	Retry logic, timeout handling, failed-job behavior, and logs.
Testing	Unit tests for job lifecycle, executor handlers, and validation.
Code quality	Small functions, readable code, minimal global state, and maintainable structure.
Documentation	README, setup steps, API examples, and demo instructions.
Demo ability	Can run server + agent and show a job moving from pending to completed.

Starter Project Included

The starter project contains a compilable Go module with basic server and agent binaries, in-memory store, executor handlers, examples, README, and test scaffolding. You are expected to extend it rather than start from an empty folder.

- cmd/server/main.go - starts the TaskBridge server.
- cmd/agent/main.go - starts the TaskBridge agent.
- internal/model - shared job and agent models.
- internal/store - concurrency-safe in-memory storage.
- internal/server - REST API handlers.
- internal/executor - safe job execution handlers.
- internal/agent - polling agent client.
- examples - sample JSON job payloads.

Deliverables

1. Public Github repo containing the final code
2. Create a demo video of 1-2 minutes showcasing the demo part of the submission and attach the link of same in Github repo
3. Screenshot of the console for each milestone where log clearly show that the milestone was achieved and attach it in the repo

Helpful Resources

1. Free Golang course for Beginners: https://www.boot.dev/courses/learn-golang